

<호텔 금고>

-1일차 과제B-

팀명 : 홍박사와 아이들

팀원 : 박규현, 임송주, 송성혁, 홍순현

< 프로그램 개요 >

이 프로그램은 ATtiny85와 TM1650 모듈을 이용해 4자리 비밀번호 금고를 구현한 것이다. 키패드 입력으로 비밀번호를 확인하고, TM1650 7세그먼트로 입력 상태·오답·잠금 시간을 표시한다. 부저와 LED는 타이머 인터럽트 기반으로 제어되어, 입력음·오답음·성공 멜로디, LED 점등 등이 비동기적으로 동작한다.

비밀번호를 잘못 입력하면 세그먼트가 깜빡이며 오답음을 내고, 4회 이상 연속 실패 시 10초간 잠금 상태가 되어 디지털 카운트다운을 표시한다. 비밀번호가 맞으면 애니메이션과 멜로디로 성공을 알리며 LED가 일정 시간 켜지는 금고 예제이다.

1. PIN & KEY 정의부

: 하드웨어 회로도 와 코드 사이를 연결해 주는 “매핑 테이블” 역할.

SDA_PIN, SCL_PIN, BUZZER_PIN, LED_PIN

- ATtiny85의 각 핀을 TM1650(SDA/SCL), 부저, 외부 LED와 연결하기 위한 심볼 정의.

KEY_ENTER, KEY_SET, KEY_RESET, KEY_BACK, KEY_NONE

- TM1650 키패드로부터 읽어온 값들을 내부에서 의미 있게 다루기 위한 논리 키값.

```
// ===== PIN & KEY =====  
// 핀 매핑  
#define SDA_PIN    PB0  
#define SCL_PIN    PB2  
#define BUZZER_PIN PB3  
#define LED_PIN     PB4  
  
#define SDA_PORT    PORTB  
#define SDA_DDR     DDRB  
#define SDA_PINREG  PINB  
  
#define SCL_PORT    PORTB  
#define SCL_DDR     DDRB  
  
// 키 매핑 (기능키)  
#define KEY_ENTER   10  
#define KEY_SET     11  
#define KEY_RESET   12  
#define KEY_BACK    13  
#define KEY_NONE    -1
```

2. 소프트웨어 I2C 드라이버 (TM1650 통신부)

: ATtiny85에는 하드웨어 I2C 모듈이 없으므로, GPIO를 직접 제어하여 I2C를 구현.

`static void i2c_delay(void)`

- I2C 타이밍 확보를 위한 짧은 지연 함수.

`SDA_release(), SDA_low()`

- SDA 핀을 입력(풀업, High-Z) 상태 또는 출력 Low 상태로 제어.

`SCL_high(), SCL_low()`

- SCL 핀을 High-Z+풀업 또는 Low로 제어해 클럭을 생성.

`uint8_t SDA_read(void)`

- SDA 라인 입력값 읽기.

`void i2c_init(void)`

- SDA/SCL 초기 상태 설정.

`void i2c_start(void), void i2c_stop(void)`

- I2C START/STOP 조건 생성.

`uint8_t i2c_write_byte(uint8_t data)`

- 한 바이트를 MSB부터 전송하고, 마지막에 슬레이브 ACK를 읽음.

`uint8_t i2c_read_byte(uint8_t ack)`

- 슬레이브로부터 1바이트를 읽고, 마지막에 ACK/NACK을 전송.

```
static uint8_t i2c_write_byte(uint8_t data) {
    // MSB부터 전송
    for (int8_t i = 7; i >= 0; i--) {
        SCL_low();
        if (data & (1 << i)) SDA_release();
        else SDA_low();
        i2c_delay();
        SCL_high();
        i2c_delay();
    }
    // ACK 입력
    SCL_low();
    SDA_release();
    i2c_delay();
    SCL_high();
    i2c_delay();
    uint8_t ack = !SDA_read();
    SCL_low();
    return ack;
}
```

```
static uint8_t i2c_read_byte(uint8_t ack) {
    uint8_t data = 0;
    SDA_release();
    for (int8_t i = 7; i >= 0; i--) {
        SCL_low();
        i2c_delay();
        SCL_high();
        i2c_delay();
        if (SDA_read()) data |= (1 << i);
    }
    // ACK/NACK 전송
    SCL_low();
    if (ack) SDA_low();
    else SDA_release();
    i2c_delay();
    SCL_high();
    i2c_delay();
    SCL_low();
    SDA_release();
    return data;
}
```

3. TM1650 & 키패드 제어부

3.1) 상수 / 테이블

: TM1650을 이용한 4-digit 7세그먼트 디스플레이 출력과 4x4 키패드 입력 처리를 담당하는
중간 계층.

TM1650_CTRL_WRITE, TM1650_DIGx_WRITE

- TM1650 제어레지스터 및 각 자리(DIG1~4)의 I2C 주소.

seg_table[11]

- 숫자 0~9와 OFF에 대응하는 세그먼트 패턴값.

digit_addr[4]

- 각 자리(DIG1~4)에 대응하는 TM1650 주소 배열.

KeyMap 구조체 + keymap[]

- TM1650이 내보내는 7비트 키코드(code7)를 내부 논리 키값(0~9, SET, ENTER, RESET, BACK)으로 변환하기 위한 매핑 테이블.

3.2) 핵심 함수

void tm1650_write_raw(uint8_t addr, uint8_t pattern)

- I2C를 통해 특정 자리 주소에 세그먼트 패턴 1바이트를 직접 전송.

void tm1650_ctrl_init(void)

- TM1650 제어 레지스터에 0x8F를 써서 디스플레이 ON + 최대 밝기 설정.

void tm1650_clear_all(void) / seg_all_off()

- 4자리 모두 OFF.

int8_t keycode_to_val(uint8_t code7)

- TM1650 키코드를 내부 키 값으로 변환.

int8_t tm1650_scan(void)

- TM1650 키 레지스터 4개(0x49,0x4B,0x4D,0x4F)를 순차적으로 읽어, 눌린 키가 있으면 해당 키를 논리 값(0~9, ENTER, SET 등)으로 반환.

```

// ===== TM1650 & KEYPAD =====
// TM1650 세그먼트 주소
#define TM1650_CTRL_WRITE 0x48
#define TM1650_DIG1_WRITE 0x68
#define TM1650_DIG2_WRITE 0x6A
#define TM1650_DIG3_WRITE 0x6C
#define TM1650_DIG4_WRITE 0x6E

// 0~9 + OFF 세그먼트 패턴
static const uint8_t seg_table[11] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x6F,
    0x00
};
#define SEG_OFF 0x00

// 자리 주소 테이블
static const uint8_t digit_addr[4] = {
    TM1650_DIG1_WRITE, TM1650_DIG2_WRITE,
    TM1650_DIG3_WRITE, TM1650_DIG4_WRITE
};

// TM1650 키 레지스터 스캔
static int8_t tm1650_scan(void) {
    const uint8_t regs[4] = {0x49, 0x4B, 0x4D, 0x4F};
    for (uint8_t i = 0; i < 4; i++) {
        i2c_start();
        i2c_write_byte(regs[i]);
        uint8_t raw = i2c_read_byte(0);
        i2c_stop();

        uint8_t code7 = raw & 0x7F;
        if (code7 & 0x40) {
            int8_t k = keycode_to_val(code7);
            if (k != KEY_NONE) return k;
        }
    }
    return KEY_NONE;
}

```

4. 부저 / LED / 타이머 기반 비동기 사운드 시스템

: 부저와 LED를 완전히 인터럽트 기반 비동기 방식으로 운용해서, 메인 루프는 키 입력/상태 처리에만 집중할 수 있게 설계.

4-1) 주요 전역 변수

alarm_active, alarm_ms

- 현재 재생 중인 알람(삐 소리 등)의 남은 시간 관리.

audio_active, tone_half_us, audio_acc_us

- 부저 토글 주기(반주기 μ s)와 누적 시간 관리. 특정 주파수 톤을 만드는 데 사용.

led_ms

- LED를 켜두는 남은 시간 (ms 단위).

melody_active, melody_index

- 성공 멜로디(삐-비--빅) 재생 여부 및 현재 단계 인덱스.

4-2) 상수

TONE_HALF_US_KEY(500Hz), TONE_HALF_US_WRONG(~714Hz),
, TONE_HALF_US_OK1~3

- 키 입력음, 오답음, 성공 멜로디 각 단계의 주파수 지정.

4-3) 구조체 및 테이블

ToneStep { duration_ms, half_us, audio_on }, success_melody[]

- 성공 시 재생되는 “삐-비--빅!” 멜로디를 시간 + 주파수 + 음/침 여부로 정의한 시퀀스.

초기화 함수

buzzer_init(), led_init()

- 부저/LED 핀을 출력으로 설정하고 초기 상태를 OFF로 설정.

timer0_init_1ms()

- Timer0을 1ms 주기의 CTC 모드로 설정 (잠금 타이머, LED 타이머, 알람 타이머용).

timer1_audio_init_100us()

- Timer1을 100μs 주기 인터럽트로 설정 (부저 토글용).

4-4) 알람/멜로디 제어 함수

trigger_alarm_with_half_period(ms, half_us)

- 특정 주파수의 삐 소리를 지정된 시간(ms) 동안 재생하도록 설정.

trigger_alarm(ms)

- 기본 키 입력음(500 Hz).

trigger_alarm_fail(ms)

- 오답음 (~714 Hz).

start_success_melody() / melody_next_step()

- 성공 멜로디 전체를 시퀀스로 재생하는 상태머신.

```
// 성공 멜로디 시작
static void start_success_melody(void) {
    if (lockout_active) return;

    melody_active = 1;
    melody_index = 0;

    const ToneStep *st = &success_melody[0];
    alarm_ms = st->duration_ms;
    alarm_active = 1;

    if (st->audio_on) {
        audio_active = 1;
        tone_half_us = st->half_us;
        audio_acc_us = 0;
    } else {
        audio_active = 0;
        buzzer_off_pin();
    }
}
```

```
// 멜로디 다음 단계
static void melody_next_step(void) {
    if (!melody_active) return;

    melody_index++;
    if (melody_index >= SUCCESS_MELODY_LEN) {
        melody_active = 0;
        alarm_active = 0;
        audio_active = 0;
        buzzer_off_pin();
        return;
    }

    const ToneStep *st = &success_melody[melody_index];
    alarm_ms = st->duration_ms;
    alarm_active = 1;

    if (st->audio_on && !lockout_active) {
        audio_active = 1;
        tone_half_us = st->half_us;
        audio_acc_us = 0;
    } else {
        audio_active = 0;
        buzzer_off_pin();
    }
}
```

5. 비밀번호 / 상태 관리 모듈

: “현재 비밀번호”, “지금 사용자가 치고 있는 입력값”, “현재 모드(설정/일반)” 같은 논리 상태를 통합 관리.

5-1) 전역 변수

password[4]

- 현재 설정된 4자리 비밀번호 (초기값 0000).

input_buf[4], input_idx

- 사용자가 키패드로 입력 중인 4자리 버퍼 및 입력 길이.

mode (MODE_NORMAL / MODE_SET)

- 평소 비밀번호 확인 모드인지, 비밀번호 설정 모드인지 구분.

failed_attempts

- 연속 실패 횟수. 4회 이상일 때 잠금(start_lockout).

```
static uint8_t password[4] = {0, 0, 0, 0}; // 현재 비밀번호
static uint8_t input_buf[4] = {0, 0, 0, 0}; // 입력 버퍼
static uint8_t input_idx = 0;
static uint8_t mode = MODE_NORMAL; // 일반/설정 모드
static uint8_t failed_attempts = 0; // 누적 실패 횟수
```

6. 세그먼트 표시 도우미 (Segment Helpers)

: 비밀번호 입력 중 화면 표현을 일관성 있게 해 주는 헬퍼 레이어. 나중에 표시 방식이 바뀌더라도, 상위 로직은 이 함수만 수정하면 되도록 분리.

6-1) 함수

seg_all_off()

- 4자리 모두 OFF.

seg_show_input_only()

- input_buf[]에 들어 있는 숫자만 왼쪽부터 순서대로 표시, 나머지 자리는 OFF.

```
// 현재 입력값만 표시
static void seg_show_input_only(void) {
    for (uint8_t i = 0; i < 4; i++) {
        uint8_t pat = (i < input_idx) ? seg_table[input_buf[i]] : SEG_OFF;
        tm1650_write_raw(digit_addr[i], pat);
    }
}
```

7. 잠금 기능 & 타이머 인터럽트(ISR)

: 잠금 시간(10초 카운트다운), LED ON 유지 시간, 사운드/멜로디 시간을 모두 타이머 인터럽트에서 공통적으로 관리.

7-1) 잠금 관련 전역

lockout_active, lockout_sec, lockout_tick, prev_lockout_sec

- 잠금 여부, 남은 초(10→0), 1ms 카운터, 이전 초 표시값 관리.

7-2) 함수

start_lockout()

- 오답 4회 이상 시 호출. 잠금 플래그 set, 10초 카운트다운 초기화, 부저, LED, 디스플레이

모두 리셋

7-3) 인터럽트 핸들러

ISR(TIM1_COMPA_vect)

- 100 μ s마다 호출, audio_active일 때 누적 시간 audio_acc_us를 증가시키고, tone_half_us를 넘으면 부저 핀을 토글해 원하는 주파수의 사운드를 생성.

ISR(TIM0_COMPA_vect)

- 1ms마다 호출, 잠금 상태일 때는 lockout_tick을 통해 1초마다 lockout_sec 감소
LED 유지시간 led_ms 감소 및 자동 OFF 알람/멜로디의 남은 시간 alarm_ms를 관리하고, 스텝이 끝나면 melody_next_step() 호출

```
static void start_lockout(void) {  
    lockout_active = 1;  
    lockout_sec = 10;  
    lockout_tick = 0;  
    prev_lockout_sec = 0xFF;  
  
    alarm_active = 0;  
    audio_active = 0;  
    melody_active = 0;  
    buzzer_off_pin();  
    led_ms = 0;  
    led_off();  
    seg_all_off();  
}
```

8. 디스플레이 효과 (오답 깜빡임 & 정답 애니메이션)

: 잘못 입력했을 때는 “내가 방금 친 숫자들”만 반짝반짝 하게 해서 피드백 제공.

비밀번호를 맞췄을 때는 멜로디 + 애니메이션 + LED로 시각/청각적인 성공 피드백을 제공.

8-1) 함수

refresh_display()

- 잠금 상태가 아니면 현재 입력값만 세그먼트에 반영.

reset_input()

- 입력 인덱스를 0으로 초기화하고 화면 갱신.

blink_wrong_input()

- 오답일 때 현재 입력된 글자들만 3번 깜빡이도록 구현
(입력 자리만 켜다가 → 전부 OFF → 반복).

success_animation()

- 정답일 때 세그먼트에 '0'이 DIG1 → DIG4로 달리는 애니메이션을 2바퀴 실행.

```
static void success_animation(void) {
    if (lockout_active) return;

    for (uint8_t loop = 0; loop < 2; loop++) {
        for (uint8_t pos = 0; pos < 4; pos++) {
            for (uint8_t i = 0; i < 4; i++) {
                tm1650_write_raw(
                    digit_addr[i],
                    (i == pos) ? seg_table[0] : SEG_OFF
                );
            }
            _delay_ms(100);
        }
    }

    seg_all_off();
}
```

9. 키 입력 처리 상태 머신 (handle_key())

: “키가 눌렸을 때 어떤 논리적 행동을 할 것인가?”를 전부 담당하는 상태 머신.

9-1) 함수

- static void handle_key(int key)

9-2) 키별 동작 요약

KEY_RESET

- 입력 버퍼 리셋, 외부 LED OFF, 설정 모드 중이면 비밀번호를 0000으로 초기화하고 일반 모드로 돌아옴.

KEY_BACK

- 입력된 숫자가 있으면 한 자리 삭제.

KEY_SET

- 비밀번호 설정 모드 진입, 기존 입력 리셋, LED OFF, 안내음(삐) 출력, 숫자키 0~9 최대 4자리까지 input_buf[]에 저장하고 화면에 표시.

KEY_ENTER

- 입력이 4자리가 아니면: 바로 오답 처리

→ failed_attempts++, 오답음, 깜빡임, 필요 시 잠금(start_lockout).

- 1) 설정 모드일 때: 입력된 4자리를 새 비밀번호로 저장 후 일반 모드로 복귀, “삐삐”.

- 2) 일반 모드일 때: 입력값과 password[] 비교

- 일치 → 성공 처리

- 실패횟수 리셋

- start_success_melody() + success_animation() + LED 1.2초 켜기

- 불일치 → 오답 처리

- 실패횟수 증가

- 응답음 + 숫자 깜빡임 + 필요 시 잠금

```
static void handle_key(int key) {
    if (lockout_active) return;

    // RESET 키
    if (key == KEY_RESET) {
        reset_input();
        led_off();
        if (mode == MODE_SET) {
            // 설정 모드에서 RESET → 비밀번호 0000, 일반 모드 복귀
            password[0]=0; password[1]=0; password[2]=0; password[3]=0;
            mode = MODE_NORMAL;
            trigger_alarm(50);
        }
        return;
    }

    // BACK 키: 한 자리 삭제
    if (key == KEY_BACK) {
        if (input_idx > 0) {
            input_idx--;
            refresh_display();
        }
        return;
    }

    // SET 키: 비밀번호 설정 모드 진입
    if (key == KEY_SET) {
        mode = MODE_SET;
        reset_input();
        led_off();
        trigger_alarm(100);
        return;
    }
}
```

10. 메인 루프 (main())

- : 메인 루프는 키 스캔 + 상태에 따른 큰 흐름 제어만 담당하고,
시간에 민감한 것(부저, LED 유지, 잠금 카운트 등)은 전부 타이머 ISR에 위임한 구조.

10-1) 초기화 순서

- i2c_init(), tm1650_ctrl_init(), seg_all_off()
→ TM1650 통신 및 디스플레이 초기화.

buzzer_init(), led_init()

- 부저, LED 핀 설정.

timer0_init_1ms(), timer1_audio_init_100us()

- 타이머 설정 및 interrupt enable.

sei()

- 전역 인터럽트 허용.

10-2) 무한 루프 동작

- lockout_active == 1일 때
→ 남은 초(lockout_sec)가 변할 때마다 DIG3, DIG4에 카운트다운 표시.
(예: 10초 → __10, 9초 → ___9)
- 평상시
→ tm1650_scan()으로 키 스캔
→ 신호를 직접 polling하지 않고, “이전 프레임에는 안 눌렀고 이번 프레임에만 눌린 경우”에만 rising edge로 인식
→ 그때 trigger_alarm(40)으로 짧은 키 입력음 재생 + handle_key(key) 호출

→ 키 스캔 주기 및 디바운싱 효과를 검함.

■ 회로도



